설비 고장을 예측하는 AI 데이터 분석

시계열 분석 종합 실습

C-MAPSS FD001로 추세·변동성·상관·이상 종합과 정비 리포트

posco

CONTENTS

목차

01 데이터 준비와 시간축 정리

02 추세와 변동성 분석

ĆĊ 상관과 이상 의심 구간

04 정비 리포트와 종합 재현

posco

posco

01

데이터 준비와 시간축 정리

데이터 구조 · 시간축 변환 · 결측 보간

posco

시계열 분석 종합 실습

흩어진 시계열 기법을 하나의 흐름으로 이어붙여 이상 의심 구간 리포트까지 완성

핵심 메시지

흩어진 시계열 기법을 하나의 흐름으로 연결

설비 이상 의심 구간을 찾는 리포트 완성

종합 실습의 4단계 분석 흐름

데이터 준비 → 추세·변동성 → 상관·이상구간 → 정비 리포트

STEP 1

데이터 준비

적재 · 엔진 추출 · 센서 선별 · 시간축 정리 · 결측 보간

STEP 2

추세 변동성

이동평균·이동표준편차로 추세와 변동성 추출

STEP 3

상관·이상구간

여러 센서가 함께 가리키는 이상 의심 구간 좁히기

STEP 4

정비 리포트

사람이 읽고 행동할 정비 판단 리포트의 마침표

NASA C-MAPSS FD001 — 비행기 엔진 시뮬레이션을 수업용으로 다듬은 데이터

오늘 수업의 성격

도구 수집이 아닌 작업대에 펼쳐 물건을 만드는 단계

01 · 작업대 단계

도구를 모으는 단계가 아닌 물건을 만드는 단계

작업대에 도구를 펼쳐놓고 실제 결과물을 만들어 내는 시간

ĆČ ĭ 학습 초점

새 문법 암기가 아닌 순서와 판단에 집중

이미 손에 익은 도구를 어떤 순서로 어떻게 이어붙일지가 핵심

03 · 좋은 분석

정답은 하나가 아닌 근거 있는 판단

자신의 판단에 근거를 댈 수 있으면 좋은 분석

posco

C-MAPSS FD001 데이터 구조

가로줄 하나가 한 엔진의 한 시점 측정값 묶음

가로줄 한 개 = 한 엔진의 한 시점 측정값 묶음

가장 중요한 두 열은 엔진 번호와 사이클 번호

엔진은 1~5번, 사이클은 처음 가동부터 고장까지의 시간 역할

행은 시점, 열은 측정 항목 — 엔진 번호·사이클 번호가 분석의 두 축

posco

데이터 컬럼 그룹별 의미

식별·시간·센서 — 21개 센서 그룹이 엔진 상태 변화의 단서

그룹	컬럼	의미
식별	`unit_nr`	엔진 번호 — 1~5번 중 어느 엔진인지 식별
시간	`time_cycles`	사이클 순번 — 엔진이 몇 번째로 돌았는지
센서	`s_1~s_21`	측정값 — 온도·압력·회전수 등 21개 물리량

> FD001에서 운영 설정 3개는 거의 변하지 않아 표에서 생략

posco

RUL과 정상에서 고장까지의 흐름

모든 엔진이 결국 고장 — 정상에서 고장까지의 이야기가 한 엔진에 담김

STEP 1

정상

처음 가동, 센서값이 안정적으로 유지되는 멀쩡한 상태

ǐ i ĝ ĩ č

열화

부품이 닳고 성능이 떨어지면서 센서값이 변하기 시작 — 오늘 찾을 이상 의심 구간

STEP 3

고장

마지막 사이클에서 한계 도달 — 한 엔진의 마침표

한 엔진을 처음부터 끝까지 따라가면 정상에서 고장까지의 한 편의 이야기

posco

RUL의 의미와 분석의 안전장치

RUL은 지금 시점에서 고장까지 남은 사이클 수

RUL = 지금 시점에서 고장까지 남은 사이클 수

192 사이클에서 고장 난 엔진의 100번째 시점 RUL = 92

데이터 뒷부분으로 갈수록 이상 신호가 강해짐

오늘 찾을 이상 의심 구간의 정체

답의 큰 방향을 알고 검증 가능 — 연습용으로 적합

오늘 RUL을 직접 예측하지는 않고 다음 과정에서 다룸

posco

분석 대상 센서 선정 기준

21개 센서 중 시간에 따라 변하는 센서만 선별

21개 센서 중 시간에 따라변하는 센서만 선별

판단 기준은 표준편차 — 0에 가까우면 변화 없는 센서

표준편차가 커도 추세 없이 노이즈만 크면 그래프로 재확인

값이 변하지 않는 센서로는 망가지는 신호를 읽어낼 수 없음

posco

죽은 센서와 살아있는 센서

표준편차로 가르는 센서 선별 — 살아있는 센서 중심 분석

구분	표준편차	활용
죽은 센서 s_1, s_5, s_6 등	0에 가까움	❌ 분석에서 제외
살아있는 센서 s_2, s_4, s_7, s_11 등	큼	✅ 상태 변화 읽기에 활용

살아있는 센서를 중심으로 오늘의 모든 분석 진행

posco

실습 1. 데이터 적재와 단일 엔진·센서 선별

엔진 한 대를 추출하고 살아있는 센서만 골라내기

목표

데이터를 불러와 한 엔진을 추출하고 살아있는 센서만 골라내기

단계

· 파일을 불러와 엔진별 행 수 확인

· 2번 엔진만 추출해 인덱스 정리

· 표준편차가 0에 가까운 죽은 센서를 빼고 살아있는 센서만 남기기

예상 결과

2번 엔진 287행, 살아있는 센서 14개

posco

실습 1. 데이터 적재와 단일 엔진·센서 선별 정답

엔진 한 대를 추출하고 살아있는 센서만 골라내기

CODE · 실습 1

```
import pandas as pd
df = pd.read_csv('cmapss_fd001_sample.csv')
e2 = df[df['unit_nr'] == 2].reset_index(drop=True)
print('엔진2 행:', len(e2))                       # 287
sc = [f's_{i}' for i in range(1, 22)]
stds = e2[sc].std()
print('살아있는 센서:', int((stds > 0.01).sum()))   # 14
```

posco

cycle를 timestamp로 바꾸는 이유

날짜시간이 있으면 시간 단위 집계와 조회 가능

사이클 번호는 순서만 나타내는 숫자

날짜시간이 있으면 시간 단위 집계와 조회 가능

C-MAPSS는 시간 정보가 없어 변환 샘플을 따로 사용

1 사이클을 1시간으로 가정해 변환한 `engine1_timestamp.csv` 사용

posco

사이클 기반과 날짜시간 기반

두 기반 모두 핵심 분석엔 충분 — 시간 단위 집계는 날짜시간만 가능

구분	가로축	가능 분석
사이클 기반	**순번 (1, 2, 3 …)**	추세 · 변동성 · 상관 — 핵심 분석에 충분
날짜시간 기반	**실제 시각 (2024-01-01 …)**	위 분석 + **시간 단위 집계**와 특정 시각 조회

오늘은 두 가지 모두 연습 — 사이클 기반 + 날짜시간 기반 변환과 리샘플링

실습 ČBPÒÖÑŒPMÖŐ 변환과 시간 집계

가상 시각을 인덱스로 만들고 시간 단위로 집계

목표

사이클 데이터에 붙인 가상 시각을 인덱스로 만들고 시간 단위로 집계

단계

· 시각 열을 시간 형식으로 바꿔 인덱스로 올리기

· 6시간 단위로 리샘플링해 평균 구하기

· 집계 후 행 수로 묶인 정도 확인

예상 결과

6시간 집계 32행

posco

실습 2. timestamp 변환과 시간 집계 정답

가상 시각을 인덱스로 만들고 시간 단위로 집계

CODE · 실습 2

```
ts = pd.read_csv('engine1_timestamp.csv')
ts['timestamp'] = pd.to_datetime(ts['timestamp'])
ts = ts.set_index('timestamp')
ts_6h = ts.resample('6h').mean()
print('6시간 집계 행:', len(ts_6h))              # 32
```

실습 3. 측정 누락 보간과 데이터 확정

결측 보간·정렬·주기 재구성으로 분석용 데이터 확정

목표

결측이 있는 데이터를 보간하고 정렬·주기 재구성으로 분석용 데이터를 확정

단계

· 결측 데이터를 불러와 채우기 전 결측 수 확인

· 보간으로 채우고 정렬·1시간 간격으로 재구성

· 재구성 후 빈 시점도 보간해 결측 0 확인

예상 결과

결측 15에서 0으로, 확정 데이터 192행 결측 0

posco

실습 3. 측정 누락 보간과 데이터 확정 정답

결측 보간·정렬·주기 재구성으로 분석용 데이터 확정

CODE · 실습 3

```
miss = pd.read_csv('engine1_timestamp_missing.csv')
miss['timestamp'] = pd.to_datetime(miss['timestamp'])
miss = miss.set_index('timestamp')
print('보간 전 결측:', int(miss.isna().sum().sum()))   # 15
clean = miss.interpolate().sort_index().asfreq('1h').interpolate()
print('확정 행:', len(clean), '결측:', int(clean.isna().sum().sum()))   # 192 / 0
```

02

추세와 변동성 분석

이동통계 · z-score 비교 · 급변 탐색

이동평균과 이동표준편차

종합 실습에서 두 도구를 함께 쓰는 이유

01 · 추세

이동평균

잔잔한 흔들림을 걷어내고 **큰 흐름**만 부드럽게 남김

일정 구간의 평균을 한 칸씩 옮겨가며 계산

02 · 변동성

이동표준편차

값이 얼마나 흔들리는지 **변동성**을 측정

고장이 가까워지면 센서 출렁임이 커져 이 값이 증가

이동평균은 방향을, 이동표준편차는 안정성을 — 함께 봐야 전환점을 놓치지 않음

추세와 변동성을 함께 보는 이유

두 신호가 함께 변하는 시점이 정상에서 열화로 넘어가는 전환점

멀쩡한 설비 — 이동평균 평평, 이동표준편차 작게 유지

열화 시작 — 이동평균이 기울고 **이동표준편차도 증가**

두 신호가 함께 변하는 시점 = 정상→열화 전환점

하나만 보면 전환점을 놓치기 쉬움 — 반드시 함께 보기

posco

윈도우 크기가 해석을 바꾼다

정답은 없음 — 목적에 맞춰 윈도우 크기를 선택

구간 크기를 윈도우라고 부름

작게 잡으면 민감하지만 노이즈가 많음

크게 잡으면 부드럽지만 변화에 둔감함

여러 윈도우를 함께 그려 비교 — 우리 데이터엔 보통 10~20이 적당

작은 윈도우와 큰 윈도우

윈도우 크기 비교 — 민감함 vs 부드러움

윈도우	모양	특징
작음 — 5	민감 · 들쭉날쭉	**빠른 변화 포착** · 노이즈에 휘둘림
큼 — 30	부드러움	**큰 추세 선명** · 짧은 변화에 둔감

이론은 충분 — 직접 여러 윈도우로 이동평균을 그려 차이 확인하기

posco

실습 4. 이동평균 다중 윈도우와 열화 추세

여러 창 이동평균으로 추세를 보고 열화 방향 확인

목표

여러 창 크기 이동평균으로 추세를 보고 초반·후반 값으로 열화 방향을 확인

단계

· **창 크기** 5·10·20**으로 이동평균을 각각 구하기**

· **가장 큰 창의 마지막 값으로 추세 수준 확인**

· **초반과 후반 이동평균을 비교해 열화 방향 판정**

예상 결과

큰 창 이동평균 마지막 약 49.1, **초반** 46.9**에서 후반** 49.1**로 상승**

posco

실습 4. 이동평균 다중 윈도우와 열화 추세 정답

여러 창 이동평균으로 추세를 보고 열화 방향 확인

CODE · 실습 4

```
for w in (5, 10, 20):
    e2[f'ma{w}'] = e2['s_11'].rolling(w).mean()
print('ma20 마지막:', round(e2['ma20'].iloc[-1], 2))   # 49.1
ma15 = e2['s_11'].rolling(15).mean()
print('추세:', round(ma15.iloc[14], 1), '->', round(ma15.iloc[-1], 1))   # 46.9 -> 49.1
```

posco

z-score 표준화로 같은 눈금에 두기

크기가 다른 센서를 같은 눈금에서 비교

센서마다 값 크기가 달라 직접 비교가 어려움

각 값에서 평균을 빼고 표준편차로 나눔

모든 센서가 평균 0, 표준편차 1 기준으로 통일

s_4는 1400 근처, s_11은 47 근처 — 그대로 그리면 작은 센서가 바닥에 납작함

z-score의 의미와 활용

평균에서 표준편차 몇 배 떨어졌는지를 한 숫자로 표현

ćĉ ĭ 의미

평균에서 표준편차의
몇 배 떨어졌나

z=2 → 평균보다 2σ 높음, z=-1 → 1σ 낮음

02 · 이상 기준

절댓값 3 초과
이상이 의심되는 드문 값

정규분포 기준 표준편차 3배 밖 = 0.3% 미만

ćċ ĭ 한계

추세가 강한 데이터는
이동평균 밴드가 정확

정상 열화도 이상으로 잡힐 수 있어 상황별 선택

posco

표준화 전후 값 범위

제각각이던 센서들이 변환 후 대략 -3 ~ +3 범위로

센서	변환 전	변환 후
s_4	1389 ~ 1451	**2.0 ~ 2.9**
s_7	550 ~ 555	**2.5 ~ 1.9**
s_11	46 ~ 49	**1.8 ~ 2.3**

한 그래프에 겹쳐 그려도 **공정하게 비교 가능**

실습 5. z-score 표준화 다중 센서 비교

여러 센서를 표준화해 같은 눈금에서 이상 정도 비교

목표

여러 센서를 ŖÁĖŒŃÖØŃ로 표준화해 같은 눈금에서 이상 정도를 비교

단계

ġ 대상 센서들의 평균과 표준편차로 표준화 값을 구하기

ġ 표준화 후 값 범위를 확인

ġ Ć에서 얼마나 벗어났는지로 이상 정도를 읽기

예상 결과

표준화 값 범위가 약 ÆĈBĐ에서 ČBÇ로Æ 표준편차 단위로 표현됨

posco

실습 5. z-score 표준화 다중 센서 비교 정답

여러 센서를 표준화해 같은 눈금에서 이상 정도 비교

CODE · 실습 5

```
zc = ['s_4', 's_7', 's_11', 's_21']
z = (e2[zc] - e2[zc].mean()) / e2[zc].std()
print('z 범위 s_11:', round(z['s_11'].min(), 2), round(z['s_11'].max(), 2))   # -1.74 / 2.35
```

posco

변화량과 변화율의 의미

갑자기 확 변하는 급변을 잡는 두 도구

01 · 변화량

diff

직전 시점 대비 **변화량** — 급변 포착에 유리

완만할 땐 0 근처에 머물다 급변 시 크게 솟음

02 · 변화율

pct_change

직전 대비 **비율 변화** — 크기 다른 센서 비교

센서마다 값 크기가 다를 때 더 공정한 비교

설비 이상에는 느린 열화와 갑작 고장 — 변화량은 빠른 쪽을 잡음

posco

추세 분석과 급변 분석의 보완

두 도구는 경쟁이 아니라 서로의 빈틈을 메우는 동료

느린 열화는 이동평균이, **갑작 변화**는 변화량이 포착

두 도구는 경쟁이 아니라 서로의 빈틈을 메우는 동료

급변 시점은 이상 의심 구간 후보가 됨

서서히 닳는 열화 + 갑자기 부서지는 급작 고장 — 둘 다 잡으려면 함께 보기

시계열 분해 (선택)

시계열을 추세·주기·잔차 세 갈래로 나누기

01 · 추세

길게 봤을 때
오르내리는 큰 흐름

02 · 주기

하루나 일주일처럼
반복되는 패턴

03 · 잔차

추세·주기로
설명 안 되는 나머지

잔차만 보면 이상이 도드라지기도 함

시계열 분해가 유용한 경우

주기성이 강한 데이터에서 진짜 위력 발휘

주기성이 강한 데이터에서 진짜 위력 발휘

C-MAPSS는 주기가 약해 효과가 제한적

ARIMA 등 더 깊은 기법은 이 과정 범위 밖

낮에 바쁘고 밤에 쉬는 설비처럼 하루 주기가 있을 때 효과 큼

실습 6. 급변과 변동성 밴드 이탈

변화량 급변과 이동평균 밴드 이탈을 세기

□□

변화량으로 급변을 찾고 이동평균에 표준편차 두 배 밴드 이탈을 세기

□□

- 변화량 절댓값 평균에 표준편차 세 배를 더한 급변 기준을 정하기
- 급변 기준을 넘는 시점 수를 세기
- 이동평균 밴드 상단을 넘는 점 수를 세기

□□ □□

급변 의심 1곳 밴드 상단 돌파 8곳

posco

실습 6. 급변과 변동성 밴드 이탈 정답

변화량 급변과 이동평균 밴드 이탈을 세기

CODE · 실습 6

```
d = e2['s_11'].diff()
thr = d.abs().mean() + 3 * d.abs().std()
print('급변 의심:', int((d.abs() > thr).sum()))       # 1
ma = e2['s_11'].rolling(20).mean(); sd = e2['s_11'].rolling(20).std()
print('밴드 상단 돌파:', int((e2['s_11'] > ma + 2*sd).sum()))   # 8
```

posco

03

상관과 이상 의심 구간

센서 상관 · 임계선 · 이상 종합

상관계수와 heatmap

두 센서가 함께 움직이는 정도를 하나의 숫자로

1에 가까우면 같은 방향으로 함께 움직임

1에 가까우면 반대 방향, 0에 가까우면 무관

정상적으로 함께 열화되는 센서는 강한 상관

`corr` 한 줄로 모든 쌍을 한 번에 구함

같은 엔진 부품이 닳으면 여러 센서가 일관되게 반응 ∴ 정상 열화 센서는 강한 상관

상관 해석 시 주의점

상관은 같은 이야기를 하는 센서를 알려줌 — 인과는 아님

heatmap은 상관계수를 색으로 표현한 그림

진한 색일수록 강한 관계를 나타냄

상관이 높아도 한쪽이 원인이라는 뜻은 아님

두 센서가 함께 움직이는 건 대개 엔진 열화라는 공통 원인의 결과

ACF·PACF 자기상관 개념

한 센서가 자기 과거와 얼마나 닮았는지 보는 개념 — 이 과정 범위 밖

자기상관 = 지금 값이 과거의 자기 값과 닮은 정도

오늘 값이 어제 값과 비슷하면 1칸 자기상관이 높음

ARIMA 같은 예측 모델에 쓰임 — 이 과정 범위 밖

PACF는 중간 시점 영향을 걷어내고 특정 시차만의 순수한 영향

코드 실습은 하지 않고 개념만 ㅎ 깊이 들어가지 않음

자기상관이 쓸모 있는 이유

과거가 현재를 알려주는 성질 — 이동평균이 잘 작동하는 이유

센서 값은 보통 직전 값과 비슷하게 이어짐

이 성질 덕분에 이동평균 같은 방법이 잘 작동

지금은 센서끼리의 상관에 집중

1초 전 온도가 50도면 지금도 50도 근처 — 갑자기 100도로 튀지 않음

posco

실습 7. 다중 센서 상관과 heatmap

핵심 센서 상관행렬을 heatmap으로 강한 관계 확인

목표

핵심 센서들의 상관행렬을 구해 heatmap으로 강한 관계를 확인

단계

· 핵심 센서의 상관행렬을 계산

· 특정 두 센서의 상관을 확인

· 히트맵으로 강한 양·음 관계를 색으로 파악

예상 결과

s_2와 s_15가 강한 양(0.9), 히트맵으로 관계 확인

posco

실습 7. 다중 센서 상관과 heatmap 정답

핵심 센서 상관행렬을 heatmap으로 강한 관계 확인

CODE · 실습 7

```
import seaborn as sns
import matplotlib.pyplot as plt
core = ['s_2', 's_4', 's_7', 's_11', 's_15', 's_21']
corr = e2[core].corr()
print('s_2-s_15:', round(corr.loc['s_2', 's_15'], 2))   # 0.9
sns.heatmap(corr, annot=True, cmap='coolwarm', center=0); plt.show()
```

이상 의심 구간 임계선 긋기

눈대중이 아닌 객관적 기준선

정상과 이상을 가르는 객관적 기준선이 임계선

평균에서 표준편차의 2~3배 떨어진 곳에 선을 그음

±3배를 벗어난 값은 통계적으로 아주 드문 값

정규분포 표준편차 3배 밖 = 전체의 0.3% 미만

posco

고정 임계선과 이동 밴드

값 안정성과 추세 변화 — 상황에 맞춰 고르기

방식	기준선	적합
고정 임계선	전체 평균 기준	값이 **안정적**일 때
이동 밴드	이동평균 추종	**추세가 변할 때** · 천천히 변하는 설비 데이터

표준편차 **2배**는 민감하지만 거짓 경보 ↑ · **3배**는 확실한 이상만 잡지만 약한 신호 놓침

이상 의심 구간 종합 판단

네 가지 신호가 같은 구간을 가리키면 진짜 이상



신호 하나는 우연일 수 있으나 여러 신호의 합의는 진짜 이상

posco

왜 신호의 합의가 중요한가

한 신호만 보면 거짓 경보 — 합의가 신뢰의 근거

센서 데이터에는 늘 노이즈가 섞여 있음

한 신호만 보면 거짓 경보를 남발하게 됨

서로 다른 신호가 겹치면 진짜일 확률이 높아짐

거짓 경보가 잦으면 현장이 경보를 무시 — 가장 위험한 상태

posco

실습 8. z-score 다중 센서 이상 종합

이상 신호를 합산하고 겹치는 시점으로 구간 좁히기

목표

여러 센서의 표준화 이상 신호를 합산하고 겹치는 시점으로 구간을 좁히기

단계

· 핵심 센서를 표준화 값으로 변환

· 각 시점에서 기준 2를 넘는 센서 수를 합산

· 두 개 이상 겹치는 시점을 의심 구간으로 추리기

예상 결과

최대 동시 이상 3개 센서, 두 개 이상 겹친 시점 9곳

posco

실습 8. z-score 다중 센서 이상 종합 정답

이상 신호를 합산하고 겹치는 시점으로 구간 좁히기

CODE · 실습 8

```
core = ['s_2', 's_4', 's_7', 's_11', 's_15', 's_21']
z = (e2[core] - e2[core].mean()) / e2[core].std()
flag = (z.abs() > 2).sum(axis=1)
print('최대 동시 이상:', int(flag.max()))              # 3
print('2개 이상 겹친 시점:', int((flag >= 2).sum()))    # 9
```

posco

04

정비 리포트와 종합 재현

리포트 작성 · 전체 파이프라인

posco

분석을 정비 의사결정으로 연결하기

현장 정비 담당자는 그래프가 아니라 그래서 무엇을 할지를 알고 싶어함

핵심 메시지

분석의 끝은 그래프가 아니라 행동으로 이어지는 판단

고장 확정이 아닌 이상 의심·점검 권장으로 신중하게 표현

posco

분석가 언어를 현장 언어로

같은 사실이라도 현장 언어는 행동을 알려줌

유형	같은 사실을 다르게
❌ 분석가 언어	"s_4의 z-score가 150 사이클부터 2 초과" 정확하지만 담당자는 무엇을 해야 할지 알기 어려움
✅ 현장 언어	"수명 후반 진입, 점검 일정 검토 권장" 사람이 무엇을 해야 할지 알려줌

신호 강도에 맞게 **표현 수위 조절** — 신중한 절제가 분석가의 신뢰

posco

비용과 위험의 균형

신호 강도에 맞는 균형 잡힌 조치 제안이 핵심

점검에는 비용이 들고 설비를 멈추면 생산 손실 발생

신호를 무시했다 고장 나면 훨씬 큰 손실

신호 강도에 맞는 균형 잡힌 조치 제안이 핵심

설비를 아는 사람이 데이터 분석까지 하면 훨씬 정확한 판단

리포트 5대 구성 요소

분석을 모르는 담당자도 읽고 행동할 수 있는 틀

요소	담을 내용	예시
대상·시점	엔진 · 사이클	**1번 150 이후**
관찰 신호	센서 변화	**s_4 상승 이탈**
종합 근거	신호 개수	**4개 동시**
권장 조치	행동 제안	**점검 검토**

정상 구간과의 통계 비교까지 더하면 다섯 요소가 완성

리포트 작성 시 흔한 실수

결과와 할 일만 — 분석 과정은 담당자에게 불필요

❌ 피해야 할 실수

분석 과정을 너무 자세히 쓰는 것

결론 없이 그래프만 잔뜩 붙이는 것

과장하거나 너무 모호하게 쓰는 것

"큰일 났습니다" / "뭔가 좀 이상한 것 같기도 합니다" — 둘 다 나쁨

신호 강도에 맞게 구체적이면서 절제된 톤이 좋음

KAMP 국내 제조 데이터 비교 (선택)

C-MAPSS와 구조가 비슷해 같은 분석 적용 가능

KAMP는 국내 중소 제조기업용 인공지능 제조 플랫폼

사출i용접i열처리 등 실제 공정 센서 데이터 제공

C-MAPSS와 구조가 비슷해 같은 분석 적용 가능

측정 주기와 센서 종류는 달라도 분석 틀은 동일

4단계 분석 틀의 보편성

데이터가 달라도 분석의 큰 틀은 변하지 않음

데이터가 달라도 분석의 큰 틀은 변하지 않음

익숙한 국내 데이터는 결과 해석이 더 직관적

같은 흐름을 다른 데이터에 적용하면 실력 상승

비행기 엔진은 낯설지만 사출기 온도·용접기 전류는 익숙

posco

전체 파이프라인 종단 재현 안내

오늘의 졸업 작품 — 처음부터 끝까지 혼자 완성

지금까지 단계별로 나눠 연습한 것을 하나로 합침

이번엔 안내 없이 스스로 순서를 떠올리며 진행

막힐 때는 분석 4단계 중 어디인지 자문하기

실제 현장에서는 아무도 단계를 불러주지 않음 — 완벽보다 완주

자문 — 지금 내가 분석 4단계 중 어디에 있지?

종합실습 3부 구성

1부 준비·추세 → 2부 상관·이상 → 3부 비교·공유

PART 1

준비·추세

데이터 준비부터 추세·변동성 분석까지

PART 2

상관·이상

상관 분석 + 이상 구간 종합 + 리포트

PART 3

비교·공유

정상 vs 고장 임박 비교 + 결과 공유

1부 — 데이터를 불러와 전처리 + 추세·변동성 분석부터 시작

실습 9. 종합 파이프라인 재현과 구간 대조

전 구간을 재현하고 정상·고장임박 구간을 대조

목표

적재부터 이상 종합까지 한 엔진 전 구간을 재현하고 정상·고장임박 구간을 대조

단계

· 이동평균으로 추세를 내고 표준화 이상 신호를 합산

· 앞 절반 정상 구간과 뒤 절반 임박 구간으로 나누기

· 두 구간의 이상 신호 합계를 비교해 열화 확인

예상 결과

정상 구간 이상 3건, 고장임박 구간 36건으로 급증

posco

실습 9. 종합 파이프라인 재현과 구간 대조 정답

전 구간을 재현하고 정상·고장임박 구간을 대조

CODE · 실습 9

```
core = ['s_2', 's_4', 's_7', 's_11', 's_15', 's_21']
ef = ((e2[core] - e2[core].mean()) / e2[core].std()).abs() > 2
half = len(e2) // 2
print('ma20 마지막:', round(e2['s_11'].rolling(20).mean().iloc[-1], 2))   # 49.1
print('이상신호 정상 vs 임박:',
      int(ef.iloc[:half].sum().sum()), 'vs', int(ef.iloc[half:].sum().sum()))   # 3 vs 36
```